



Scan to open

Payment Health & Recovery Engine

Detect, classify, inform, recover, alert: fixing the customer's experience of a failed payment

[View full case study online](https://praaah18.github.io/case01_paymentFailures/)

1 Document Control

Product	Google Pay, UPI Payments	Target market	India
Status	Draft / Review Ready (Proposed MVP)	Context	Aug 2026 NPCI benchmarks
Author	Prachi Rai, Product Management Intern	Version	v1.0, Oct 2026
Case study	https://praaah18.github.io/case01_paymentFailures/ (copy and paste into any browser)		

2 Strategic Context: Why Now

24.51B

UPI transactions in Aug 2026 (NPCI)

₹29.82 L Cr

UPI transaction value, Aug 2026

4

failure categories to separate and handle

Problem. At 24.51B transactions a month, even a small failure share touches millions of attempts. Bank, network and NPCI failures are outside Google Pay's direct control; the **uncertainty** they create is not. A generic "Payment Failed" does not say whether money was debited or whether to retry, which damages trust and drives retries, duplicate-payment risk and support contacts.

Core principle. *Don't just fix the failure. Fix the customer's experience of the failure.* **Persona:** active UPI users making high-intent payments (merchants, bills, P2P) who hit sudden transaction ambiguity.

Failure state	What it is	Required product behaviour
Business	User/account-side: wrong PIN, low balance, limit exceeded	Explain the reason; guide the fix
Technical	Bank downtime, network failure, timeout, NPCI issue	Inform, block unsafe retry, monitor, alert team
Pending / Unknown	Confirmation delayed; status not yet known	"We couldn't confirm"; check status before retry
Debit, no completion	Debited, but payment not completed	State the debit, track reversal, notify

3 MVP Scope vs. Explicitly Out of Scope

IN SCOPE: MVP (5 components)

- **1. Failure classification:** routes failures into the 4 states.
- **2. Contextual customer notifications:** state-specific copy with next step.
- **3. Transaction status check:** poll until final state; safe retry only.
- **4. Internal anomaly alerts:** flag a bank breaching its baseline.
- **5. Basic incident dashboard:** users, transactions, value, bank health.

OUT OF SCOPE: POST-MVP

- **Alternative routing** to another linked account.
- **Customer-facing bank health warnings** before payment.
- **Advanced preventive validation.**
- *Non-goals:* rebuilding bank/NPCI rails; auto-retry of uncertain payments; weaker fraud checks.

4 Metrics & Guardrails

North Star

First-Attempt Payment Success Rate = successful first-attempt transactions ÷ total payment attempts**Supporting:** Payment Recovery Rate, Technical Failure Rate, Customer Resolution Time, Support Ticket Volume, MTTD, MTTR.

Guardrail	Rule	Breach response
Duplicate payments	No retry path exposed for UNKNOWN / PENDING	Launch blocker; pause rollout
Fraud rate	Fraud and authentication checks never weakened to lift success; track false-positive blocks	Revert the change
Payment latency	Classification adds ≤ 50 ms to checkout (<i>proposed; validate with Eng</i>)	Optimise before scaling
Support ticket volume	Payment-related contacts must trend down, not up	Review message copy

5 Functional Requirements: User Stories & Acceptance Criteria

ID / Pri.	User story	Requirement	Acceptance criteria (pass / fail)
FR-1 P0	Confirmed Failure. As a user whose bank rejected the payment, I want to know it is safe to try again, so I can complete my purchase immediately.	When the bank/NPCI returns a definitive FAILED status, show an explicit failure state with a safe-retry action.	• UI shows "Your bank declined the transaction". • "Retry Payment" button is visible and enabled. • Copy "You can safely try again" is visible.
FR-2 P0	Unknown status. As a user facing a bank timeout, I want to be blocked from accidentally paying twice, so I do not lose duplicate funds.	When status is UNKNOWN , lock the screen into a monitoring state and poll status asynchronously until a final state is reached.	• Primary CTA is disabled or hidden; no retry possible. • Status check API invoked every 15 s (<i>proposed; bounded by NPCI/PSP limits, see Q1</i>). • UI shows "Please don't retry until we confirm the status". • On final state, user receives a resolution notification.

Safety rules (apply to all states): never auto-retry an unknown transaction; every payment shows one clear state with *what happened* + *what to do next*; fraud and authentication controls stay intact.

6 Operations & Feedback Loop

Internal alerting. The alerting engine reads streaming payment events and compares each bank's technical-failure rate over a **rolling 5-minute window** against its **7-day baseline** (*proposed windows*). A breach opens an incident and routes it to the on-call schedule (e.g. PagerDuty). Exact threshold multiples are calibrated in Phase 1 from real baselines. *Illustration from the case study (hypothetical): Bank X baseline 1.2% rises to 8.7%, about 7x, so an incident opens.*

Basic incident dashboard (MVP)	Detail
Metrics shown	Affected users; failed and pending transactions; failure rate vs baseline; failure type; transaction value affected; incident duration
Filters	Bank, error type, time period
Telemetry tags (every checkout step)	transaction_id, bank_routing_code, failure_type_code, resolution_latency
Feedback loop	Detect → customer informed → team alerted → root cause → engineering fix → failure rate falls → better data, better detection

Strategic open questions for engineering sync

Q1. Polling limits and cost. What API throttling limits do NPCI or partner PSP banks impose on continuous status polling of UNKNOWN transactions, and how do we balance real-time tracking against server cost?

Q2. 24-hour stuck transactions. If a transaction stays UNKNOWN or PENDING beyond 24 hours because of downstream bank failure, what automated fallback protects customer trust without forcing a manual support ticket?

Rollout (12 weeks)	Success criteria
P1 Baseline, wk 1–2 P2 Detection, wk 3–5 P3 Recovery, wk 5–8 P4 Dashboard, wk 8–10 P5 Experiment, wk 10–12: current failure experience vs new recovery flow	↑ first-attempt success, payment recovery; ↓ unnecessary retries, support contacts, time to detect; while holding fraud and duplicate-payment guardrails.

When a payment fails, the customer should never be left wondering what happened to their money.